

Ejercicio integrador
Unidad 0: Herramientas y preliminares
Eduardo Acuña Yeomans
Lenguajes de Programación
Semestre 2026-2
Lic. en Ciencias de la Computación
Universidad de Sonora
Entrega lunes 24 de agosto
Vale el 10 % de la calificación final

Este ejercicio junta lo de estas dos semanas en una sola pieza: una gramática, los **structs** que la representan, la función que la evalúa, y un trazado hecho a mano con la especificación. Es trabajo **individual**.

La gramática

```
Exp ::= Num
      |  Id                nueva
      |  add(Exp, Exp)
      |  mul(Exp, Exp)
      |  with Id = Exp in Exp  nueva
```

Las tres producciones en negro ya las trabajaron. Las dos en rojo son las que traen algo nuevo: **nombres**.

Recordatorio de qué hace **with**: en **with y = add(3, 4) in mul(y, y)** primero se calcula **add(3, 4)**, que da 7. Dentro del cuerpo el nombre **y** vale 7. Y el cuerpo **mul(y, y)** da 49.

Sección 1: representación

Define los **structs** que representan el AST de esta gramática: **uno por producción**.

Para que las pruebas corran sobre tu archivo, los **structs** tienen que llamarse así y tener este número de campos:

Producción	struct
Num	num-exp, 1 campo
Id	id-exp, 1 campo
add(Exp, Exp)	add-exp, 2 campos
mul(Exp, Exp)	mul-exp, 2 campos
with Id = Exp in Exp	with-exp, 3 campos

Los tres campos de `with-exp` van en este orden: el nombre, la expresión que se le liga, y el cuerpo donde ese nombre se usa.

Los nombres de los *campos* son tuyos: puedes ponerles como quieras. Define los cinco `structs` con la opción `#:transparent`, para poder verlos impresos en el REPL.

Un nombre se escribe con comilla simple: `(id-exp 'y)`. La comilla simple dice que `y` es un nombre del lenguaje que estamos representando, no una variable de Racket.

Sección 2: la función `calc`

Escribe `(calc e asocs)`, que recibe una expresión del AST y una lista de asociaciones nombre $\rightarrow$ valor, y devuelve el número que resulta. Úsala con `match`, un caso por variante.

Las tres operaciones sobre las asociaciones te las doy hechas. Cópialas tal cual a tu archivo:

```
(define (asocs-vacias) '())

(define (extender asocs nombre valor)
  (cons (cons nombre valor) asocs))

(define (buscar nombre asocs)
  (cdr (assoc nombre asocs)))
```

De los cinco casos de `calc`, tres ya los tienes y solo hay que pasarles `asocs` hacia abajo. `id-exp` es el único que *consulta* las asociaciones, y `with-exp` el único que las *cambia*.

Casos de verificación

Construye estos cuatro árboles a mano y comprueba que `calc` da lo que dice la última columna. Se evalúan con las asociaciones vacías: `(calc caso (asocs-vacias))`.

Expresión	Resultado
(a) <code>add(3, mul(2, 4))</code>	11
(b) <code>with y = add(3, 4) in mul(y, y)</code>	49
(c) <code>with x = add(2, 3) in with y = mul(x, 2) in add(x, y)</code>	15
(d) <code>with a = 5 in mul(with b = add(a, 1) in b, a)</code>	30

Fíjate en (d): un `with` puede aparecer donde vaya cualquier expresión, no solo al principio.

Estos cuatro son para que te verifiques tú. Las pruebas con las que califico son más amplias, pero no usan nada que no esté en la gramática de arriba.

Sugerencia: cómo se construye un árbol a mano

El caso (b) es este, y de ahí salen los demás:

```
(with-exp 'y  
  (add-exp (num-exp 3) (num-exp 4))  
  (mul-exp (id-exp 'y) (id-exp 'y)))
```

Sección 3: el trazado

Esta es la especificación de `calc`. Es la misma función de la sección 2, escrita como definición por casos en lugar de como código:

```
[num]  (calc (num-exp n) σ)      = n
[id]    (calc (id-exp x) σ)      = lookup(x, σ)
[add]   (calc (add-exp e1 e2) σ) = (calc e1 σ) + (calc e2 σ)
[mul]   (calc (mul-exp e1 e2) σ) = (calc e1 σ) × (calc e2 σ)
[with]  (calc (with-exp x e1 e2) σ) = (calc e2 extend(σ, x, (calc e1 σ)))
```

σ es lo que en el código se llama `asocs`. `lookup` es buscar y `extend` es extender. Misma cosa, otro nombre.

Traza esta expresión, empezando con σ_0 vacío:

```
with x = add(2, 3) in with y = mul(x, 2) in add(x, y)
```

Reglas del formato:

- Un renglón por paso, y a la derecha el **nombre de la regla que aplicaste**: `[num]`, `[id]`, `[add]`, `[mul]` o `[with]`.
- Una sola regla por renglón. No juntes dos reglas distintas en el mismo paso.
- Las sustituciones van escritas. No saltes del principio al resultado.
- Puedes abreviar una subexpresión con una letra, diciendo qué es. Lo que no puedes saltarte es el paso donde σ se extiende.

Este es el formato, con el ejemplo que ya vimos en clase. El tuyo lleva dos `with`, así que σ se extiende dos veces:

```
#!/
Trazado de: with y = add(3, 4) in mul(y, y)

Abreviaturas:
N3 = (num-exp 3)      N4 = (num-exp 4)
A  = (add-exp N3 N4)
C  = (mul-exp (id-exp y) (id-exp y))

(calc (with-exp y A C) sigma0)
= (calc C extend(sigma0, y, (calc A sigma0)))
= (calc C extend(sigma0, y, (calc N3 sigma0) + (calc N4 sigma0))) [with]
= (calc C extend(sigma0, y, 3 + 4)) [add]
= (calc C sigma1) con sigma1 = extend(sigma0, y, 7) [num]
= (calc (id-exp y) sigma1) * (calc (id-exp y) sigma1) [mul]
= lookup(y, sigma1) * lookup(y, sigma1) [id]
= 7 * 7
= 49
!#
```

Qué entregas y por dónde

Un solo archivo, `integrador.rkt`, con las tres secciones en orden. Súbelo a tu repositorio del curso en GitHub, en la carpeta `unidad-0/`, **antes del lunes 24 de agosto**.

El trazado va dentro del mismo archivo, en un comentario de bloque `#/ ... /#`, como el modelo de arriba. Escribe σ como `sigma0`, `sigma1`, `sigma2`.

Antes de subirlo: abre el archivo en DrRacket y corre los cuatro casos de verificación. Si el archivo no compila, el primer criterio no se puede evaluar.

Cómo se califica

Tres criterios. Están completos aquí. Nada se califica con algo que no esté en esta hoja.

Criterio 1: código funcional

- ✓ El código compila. Los `structs` corresponden a la gramática (uno por producción). `calc` produce resultados correctos para las pruebas.
- ✗ No compila, o falla en más de un caso de las pruebas, o los `structs` no corresponden a la gramática.

Criterio 2: corrección del trazado

- ✓ El trazado llega al resultado correcto. Cada paso aplica una regla de la especificación y la identifica. Las sustituciones son explícitas.
- ~ Resultado correcto, pero los pasos están incompletos o no se identifican las reglas.
- ✗ Resultado incorrecto, o trazado ausente, o no usa las reglas de la especificación.

Criterio 3: `with`

- ✓ `calc` extiende la lista de asociaciones en el caso `with-exp` y evalúa el cuerpo con las asociaciones extendidas. El trazado muestra esa extensión.
- ✗ La implementación es incorrecta, o el trazado no muestra la extensión.

Sugerencia: por dónde empezar

Empieza por la sección 1. Sin los `structs` correctos, las otras dos no se sostienen: el `match` de `calc` se escribe contra ellos y el trazado se escribe contra el árbol.

Si te atorras en `with-exp`, vuelve a la regla `[with]` y léela despacio, de adentro hacia afuera. Primero se calcula e_1 con el σ que hay. Con ese valor se extiende. Y el cuerpo e_2 se calcula con el σ *extendido*, no con el de antes. La regla dice exactamente qué escribir.